

```
In [ ]: # Structured output + eval harness – API version
        # PE6201 · Class 3. Student notebook – run top to bottom.
```

PE6201 · Class 3 — Steer it & prove it (API)

What you build: a prompt that turns a messy loan-application form into **strict JSON**, and a **10-case eval harness** that scores it. Then you improve the prompt one change at a time and watch the **pass-rate climb**.

This is the **API** notebook — same task, same harness, but a frontier-grade hosted model. The `PE6201_Class3_Extract_local` notebook does all of this with **no key and no credits**.

You need an API key. It is read from an environment variable / Colab secret, or typed with `getpass` — **never** type a key into a code cell.

Cost: this notebook makes roughly 40 short calls (4 prompt versions × 10 cases). On a small model that is a fraction of a cent — but we **count the tokens** at the end, because Capsule 3 is about exactly that.

The loop you are learning: write the eval set → run → read the failures → change ONE thing → re-run.

Setup

```
In [2]: %pip -q install openai
```

Note: you may need to restart the kernel to use updated packages.

Connect to your provider

Default is **OpenRouter** (one key, many models) — same route as the Class 1 and Class 2 notebooks.

```
In [3]: import os, getpass, json, re, time
        from openai import OpenAI

        def _get_openrouter_key():
```

```

k = os.environ.get('MY_PRIVATE_OPENROUTER_KEY') or os.environ.get('OPENROUTER_API_KEY')
if k: return k
try:
    from google.colab import userdata
    k = userdata.get('OPENROUTER_API_KEY')
    if k: return k
except Exception:
    pass
return getpass.getpass('Paste your OpenRouter API key (hidden): ')

API_KEY = _get_openrouter_key()
BASE_URL = 'https://openrouter.ai/api/v1'
client = OpenAI(base_url=BASE_URL, api_key=API_KEY)

# TRY THIS (knob): swap in any model your key can access – then re-run and compare pass-rates.
# 'openai/gpt-4o-mini' (default, cheap) · 'anthropic/claude-3.5-haiku'
# 'qwen/qwen-2.5-7b-instruct' · 'deepseek/deepseek-chat' (both strong, open-weight)
GEN_MODEL = 'openai/gpt-4o-mini'
FAST_MODE = False # True -> run only the first 5 eval cases
print('key loaded, model:', GEN_MODEL)

```

key loaded, model: openai/gpt-4o-mini

One helper: send a prompt, get text back

`temperature = 0` means **deterministic** decoding — essential when comparing prompt versions, otherwise you cannot tell a real improvement from sampling noise (Capsule 1, sampling).

```

In [4]: TOKENS_USED = {'in': 0, 'out': 0} # we tally these for the Capsule-3 cost discussion

def generate(user_prompt: str, system: str | None = None, max_new_tokens: int = 400,
            temperature: float = 0.0) -> str:
    """temperature 0 = always the same answer (what you want when comparing prompts).
    temperature > 0 = the model gets creative/random. TRY THIS (knob): pass temperature=1.0."""
    msgs = ([{'role': 'system', 'content': system}] if system else []) + \
        [{'role': 'user', 'content': user_prompt}]
    r = client.chat.completions.create(
        model=GEN_MODEL, messages=msgs, temperature=temperature, max_tokens=max_new_tokens)
    if getattr(r, 'usage', None):

```

```
TOKENS_USED['in'] += r.usage.prompt_tokens
TOKENS_USED['out'] += r.usage.completion_tokens
return r.choices[0].message.content

print(generate('Reply with exactly the word: ready'))
```

ready

0 · Five words you need — *start here if you don't code*

You do not need to write a line of Python to follow this notebook. Read the plain-English explanations, look at what goes in and what comes out, and watch one number — the pass-rate — move.

Word	In plain words
Model	The AI itself. A frozen thing that takes text in and gives text back. You cannot change it — only what you send it.
Prompt	Everything you send the model: your instruction, plus any information and examples. This is the part you control.
Token	The chunk of text a model reads and writes — roughly $\frac{3}{4}$ of a word. You are billed per token, in and out.
JSON	A way of writing an answer as labelled fields instead of a paragraph, so a computer can use it (explained properly below).
Eval	A test. A fixed set of examples plus the conditions a correct answer must meet — your ruler for "did this get better?"

Two more you will see in the code:

- **Assertion** — a single yes/no check, e.g. "is `monthly_income` a number?". No opinion involved.
- **Pass-rate** — how many of our 10 test cases passed. One number, comparable between attempts.

1 · The task

Real forms are messy: blank fields, dates written five ways, "80k", a currency nobody stated, and chatter from the branch officer.

What is JSON, and why do we want it?

Ask a model a question and it replies in **prose** — a nice paragraph for a human:

"The applicant, Tan Wei Ming, earns about 4,200 dollars a month and is asking for an 80k loan."

That is useless to a computer. To put it in a database or a loan system, software would have to guess where the name ends and the salary begins — and it will get it wrong.

JSON (JavaScript Object Notation) is simply a way of writing the same answer as **labelled fields** — like filling in a form instead of writing a letter:

```
{ "full_name": "Tan Wei Ming", "date_of_birth": "1994-03-14", "employer": null,  
  "monthly_income": 4200, "currency": "SGD", "loan_amount": 80000 }
```

Read it as: *label on the left, value on the right*. Text sits in "quotes", numbers do not, and `null` is the formal way of saying "this field was genuinely empty".

Why this matters — three things become possible:

1. **A program can use it.** `answer["monthly_income"]` gives you `4200`. No guessing.
2. **A test can check it.** You can ask "is income a number?" — you cannot ask that of a paragraph.
3. **Therefore you can measure the prompt.** That is the whole point of today: prose can only be admired; structure can be **scored**.

All data below is SYNTHETIC — invented for teaching. Never paste real customer or personal data into a shared notebook or a third-party API. (Week 6 covers the governance side.)

```
In [5]: SCHEMA_FIELDS = ['full_name', 'date_of_birth', 'employer', 'monthly_income', 'currency', 'loan_amount']  
ALLOWED_CURRENCIES = {'SGD', 'USD', 'CNY', 'MYR', 'INR', 'EUR', 'GBP'}  
  
SAMPLE_FORM = """LOAN APPLICATION (walk-in)  
Name : Tan Wei Ming  
DOB  : 14 Mar 1994  
Employer :  
Income  : 4,200 per month  
Requesting 80k over 5 years  
Branch note: walk-in, Tues"""
```

```
print(SAMPLE_FORM)
```

```
LOAN APPLICATION (walk-in)
Name : Tan Wei Ming
DOB  : 14 Mar 1994
Employer :
Income  : 4,200 per month
Requesting 80k over 5 years
Branch note: walk-in, Tues
```

2 · The eval set — write the ruler BEFORE you tune

Ten cases: **typical**, **edge** (missing values, odd formats, other currencies) and **adversarial** (a distractor number, two names on one form).

Each case carries `expect` — the **conditions** a correct answer must meet, not one exact wording. Freeze this set; if you keep adding cases while you tune, your pass-rate stops being comparable.

```
In [6]: from dataclasses import dataclass, field

@dataclass
class EvalCase:
    id: str
    kind: str          # typical | edge | adversarial
    note: str
    text: str
    expect: dict = field(default_factory=dict)

CASES = [
    EvalCase('c01', 'typical', 'clean form, everything present', """LOAN APPLICATION – Branch 004
Name: Priya Nair
Date of Birth: 1991-07-02
Employer: Sembcorp Industries
Monthly income: SGD 6,500
Loan requested: SGD 120,000 over 6 years""",
        {'full_name': 'Priya Nair', 'date_of_birth': '1991-07-02', 'employer': 'Sembcorp Industries',
         'monthly_income': 6500, 'currency': 'SGD', 'loan_amount': 120000}),
```

```

EvalCase('c02', 'edge', 'employer left blank -> must be null, not invented', SAMPLE_FORM,
    {'full_name': 'Tan Wei Ming', 'date_of_birth': '1994-03-14', 'employer': None,
     'monthly_income': 4200, 'currency': 'SGD', 'loan_amount': 80000}),

EvalCase('c03', 'typical', 'date written in words - very common on real forms', """"Applicant: Muhammad Faizal b
Born on 3 December 1988
Works at PSA International
Earns S$5,100 monthly
Wants to borrow S$45,000""",
    {'date_of_birth': '1988-12-03', 'monthly_income': 5100, 'currency': 'SGD', 'loan_amount': 45000}),

EvalCase('c04', 'typical', 'non-SGD currency, slashed date', """"LOAN APPLICATION
Name: Li Wenjing
DOB: 1996/05/21
Employer: Alibaba Group
Monthly income: CNY 28,000
Loan amount: CNY 300,000""",
    {'currency': 'CNY', 'monthly_income': 28000, 'loan_amount': 300000, 'date_of_birth': '1996-05-21'}),

EvalCase('c05', 'edge', 'shorthand amounts (k) and DD/MM/YYYY', """"Name: Arjun Mehta
DOB: 09/11/1990 (DD/MM/YYYY)
Employer: Grab Holdings
Income: 7.2k per month
Loan: 150k""",
    {'date_of_birth': '1990-11-09', 'monthly_income': 7200, 'loan_amount': 150000}),

EvalCase('c06', 'adversarial', 'distractor: an existing repayment figure', """"Name: Chen Xiuying
DOB: 1985-02-17
Employer: Keppel Corporation
Monthly income: SGD 9,000
Existing monthly repayment: SGD 2,400
New loan requested: SGD 200,000""",
    {'monthly_income': 9000, 'loan_amount': 200000}),

EvalCase('c07', 'adversarial', 'income stated ANNUALLY -> must be converted to monthly',
    """"Applicant Sofia Rahman, born 22 Aug 1993.
Employer: Singtel.
Annual salary: SGD 96,000.
Loan sought: SGD 60,000.""",
    {'date_of_birth': '1993-08-22', 'monthly_income': 8000, 'loan_amount': 60000}),

```

```

    EvalCase('c08', 'edge', 'noisy internal chatter around the real fields', """"--- INTERNAL ---
Officer: J. Lim | queue no 87 | printer jam, re-typed
Name: Ravi Subramaniam
DOB: 1979-12-30
Employer: Changi Airport Group
Monthly income: SGD 11,500
Requested: SGD 500,000
Note: customer asked about promo rates, referred to counter 3
--- END ---""",
    {'full_name': 'Ravi Subramaniam', 'monthly_income': 11500, 'loan_amount': 500000}),

    EvalCase('c09', 'edge', 'missing date of birth -> null', """"Name: Nurul Aisyah
DOB: (not provided)
Employer: DBS Bank
Monthly income: SGD 5,800
Loan: SGD 90,000""",
    {'date_of_birth': None, 'employer': 'DBS Bank', 'monthly_income': 5800}),

    EvalCase('c10', 'adversarial', 'two names -> use the PRIMARY applicant only', """"JOINT ENQUIRY
Primary applicant: Wong Kah Wai
Co-applicant (guarantor only): Wong Mei Ling
DOB (primary): 1987-06-08
Employer (primary): ST Engineering
Monthly income (primary): SGD 7,400
Loan requested: SGD 250,000""",
    {'full_name': 'Wong Kah Wai', 'monthly_income': 7400, 'loan_amount': 250000}),
]

print(f'{len(CASES)} eval cases | ' +
      ', '.join(f'{k}={sum(1 for c in CASES if c.kind==k)}' for k in ('typical', 'edge', 'adversarial')))
```

10 eval cases | typical=3, edge=4, adversarial=3

3 · The harness — L1 assertions

In plain words

A **harness** is just an automatic marker. You give it the model's answer and the case, and it replies **pass** or **fail** — plus *why* it failed.

An **assertion** is one yes/no rule the answer must satisfy, e.g. *"income must be a number"*. We stack a handful of them. If all pass, the case passes.

L1 checks are deterministic and machine-checkable: is it valid JSON? are the keys exactly right? is income a number? is the currency allowed? No opinion involved — which is why they are free to run and why you will actually run them. (**L2** — "is this summary faithful?" — needs a judge or a human, and we are not doing that today.)

The three functions below do exactly three things:

Function	In plain words
<code>parse_json</code>	digs the JSON out of whatever the model said (it loves to add "Sure! Here you go:")
<code>l1_checks</code>	the structural rules every answer must satisfy, whatever the form said
<code>expect_checks</code>	the rules for this particular case — e.g. "employer must be null here"

```
In [7]: def parse_json(text: str):
        """Models like to wrap JSON in prose or ```json fences. Be forgiving on the way IN,
        strict about what counts as a pass."""
        if text is None:
            return None, 'empty response'
        t = text.strip()
        t = re.sub(r'^```(?:json)?|```$', '', t, flags=re.MULTILINE).strip()
        i, j = t.find('{'), t.rfind('}')
        if i == -1 or j == -1 or j < i:
            return None, 'no JSON object found in the response'
        try:
            return json.loads(t[i:j+1]), None
        except json.JSONDecodeError as e:
            return None, f'invalid JSON: {e.msg}'

    def _is_number(v):
        return isinstance(v, (int, float)) and not isinstance(v, bool)

    def l1_checks(obj) -> list:
        """Structural rules every answer must satisfy, whatever the form said."""
        p = []
        if not isinstance(obj, dict):
```

```

        return ['response is not a JSON object']
    missing = [f for f in SCHEMA_FIELDS if f not in obj]
    extra = [k for k in obj if k not in SCHEMA_FIELDS]
    if missing: p.append(f'missing keys: {missing}')
    if extra: p.append(f'unexpected keys: {extra}')

    v = obj.get('full_name')
    if not (isinstance(v, str) and v.strip()):
        p.append('full_name must be a non-empty string')

    v = obj.get('date_of_birth')
    if v is not None and not re.fullmatch(r'\d{4}-\d{2}-\d{2}', str(v)):
        p.append('date_of_birth must be YYYY-MM-DD or null')

    v = obj.get('employer')
    if v is not None and not isinstance(v, str):
        p.append('employer must be a string or null')

    for k in ('monthly_income', 'loan_amount'):
        if not _is_number(obj.get(k)):
            p.append(f'{k} must be a plain number (no commas, no "k", no currency symbol)')

    if obj.get('currency') not in ALLOWED_CURRENCIES:
        p.append(f'currency must be one of {sorted(ALLOWED_CURRENCIES)}')
    return p

def expect_checks(obj, expect: dict) -> list:
    """Case-specific expectations – the interesting failures live here."""
    p = []
    if not isinstance(obj, dict):
        return []
    for k, want in expect.items():
        got = obj.get(k, '<missing>')
        if _is_number(want) and _is_number(got):
            if abs(got - want) > 0.01:
                p.append(f'{k}: expected {want}, got {got}')
        elif got != want:
            p.append(f'{k}: expected {want!r}, got {got!r}')
    return p

def score_one(raw_text, case: EvalCase):

```

```

obj, perr = parse_json(raw_text)
problems = [perr] if perr else ll_checks(obj) + expect_checks(obj, case.expect)
return (len(problems) == 0), problems, obj

# quick self-test of the harness itself (no model involved)
_good = '{"full_name":"A B","date_of_birth":"1990-01-01","employer":null,"monthly_income":1,"currency":"SGD","loan_
assert score_one(_good, EvalCase('t','typical','',',',{})) [0] is True
assert score_one('sorry, I cannot help', EvalCase('t','typical','',',',{})) [0] is False
print('harness self-test passed')

```

harness self-test passed

4 · Four prompt versions — one change each

In plain words

A **prompt version** is just one attempt at writing the instruction. We write four, each adding exactly **one** idea to the one before, then score all four with the same ruler.

Why one change at a time? If you change three things and the score improves, you have learned nothing you can reuse — you don't know which change did it. (This is the single most common mistake people make with prompts.)

The `SYSTEM` message below is a **system prompt**: standing instructions about *who the model is being*, sent with every call, so you don't repeat them each time.

Version	The one change	Rung (Capsule 2)
v1	plain instruction, nothing else	1 · be clear
v2	+ a strict JSON schema	4 · structured output
v3	+ two worked examples	2 · few-shot
v4	+ explicit rules for the edge cases	1 · be clear, precisely

```

In [8]: SYSTEM = 'You are a careful data-extraction assistant for a Singapore bank. You never invent values.'

def v1(form: str) -> str:

```

```

    return f"Extract the applicant details from this loan application form.\n\nFORM:\n{form}"

def v2(form: str) -> str:
    return (
        "Extract the applicant details from this loan application form.\n"
        "Return ONLY a JSON object with exactly these keys:\n"
        f" {' '.join(SCHEMA_FIELDS)}\n\n"
        f"FORM:\n{form}"
    )

EXAMPLES = """EXAMPLE 1
FORM:
Name: Lim Sook Yee
DOB: 5 Jan 1990
Employer: OCBC Bank
Monthly income: SGD 5,000
Loan: SGD 100,000
JSON:
{"full_name": "Lim Sook Yee", "date_of_birth": "1990-01-05", "employer": "OCBC Bank", "monthly_income": 5000, "curr

EXAMPLE 2
FORM:
Name: Hafiz Osman
DOB: 12/03/1985 (DD/MM/YYYY)
Employer:
Income: 3.5k monthly
Loan: 40k
JSON:
{"full_name": "Hafiz Osman", "date_of_birth": "1985-03-12", "employer": null, "monthly_income": 3500, "currency": "
"""

def v3(form: str) -> str:
    return (
        "Extract the applicant details from this loan application form.\n"
        "Return ONLY a JSON object with exactly these keys:\n"
        f" {' '.join(SCHEMA_FIELDS)}\n\n"
        f"{EXAMPLES}\n"
        f"NOW DO THIS ONE\nFORM:\n{form}\nJSON:"
    )

RULES = """RULES

```

- date_of_birth must be YYYY-MM-DD. Interpret DD/MM/YYYY (not US order).
- monthly_income and loan_amount must be plain numbers: no commas, no currency symbols, no "k".
"7.2k" -> 7200 ; "150k" -> 150000
- If income is given ANNUALLY, divide by 12 to get monthly_income.
- currency is a 3-letter ISO code. If it is not stated, infer it from context; a Singapore form is "SGD".
- If a field is not present on the form, return null. NEVER invent a value.
- Use the PRIMARY applicant only. Ignore co-applicants and guarantors.
- loan_amount is the NEW amount requested. Ignore existing repayments or outstanding debt.

""""

```
def v4(form: str) -> str:
    return (
        "Extract the applicant details from this loan application form.\n"
        "Return ONLY a JSON object with exactly these keys:\n"
        f"  {'', ' '.join(SCHEMA_FIELDS)}\n\n"
        f"{{RULES}}\n"
        f"{{EXAMPLES}}\n"
        f"NOW DO THIS ONE\nFORM:\n{form}\nJSON:"
    )

VERSIONS = [('v1', 'plain instruction', v1),
             ('v2', '+ strict JSON schema', v2),
             ('v3', '+ 2 few-shot examples', v3),
             ('v4', '+ explicit rules', v4)]

print(v4(SAMPLE_FORM)[:600], '\n...')
```

Extract the applicant details from this loan application form.

Return ONLY a JSON object with exactly these keys:

full_name, date_of_birth, employer, monthly_income, currency, loan_amount

RULES

- date_of_birth must be YYYY-MM-DD. Interpret DD/MM/YYYY (not US order).
- monthly_income and loan_amount must be plain numbers: no commas, no currency symbols, no "k".
"7.2k" -> 7200 ; "150k" -> 150000
- If income is given ANNUALLY, divide by 12 to get monthly_income.
- currency is a 3-letter ISO code. If it is not stated, infer it from context; a Singapore form is "SGD".
- If a field is not pres

...

5 · Run it — and read the failures

In plain words

`run_version` sends all 10 forms through one version of the prompt, marks each answer, and prints a `PASS` / `FAIL` line per case. The number at the end is the **pass-rate**.

Start with **v1**. Expect it to do badly — that failure is the teaching moment, not an accident. Watch *what kind* of thing goes wrong, because that tells you which fix to reach for.

```
In [9]: def run_version(builder, cases=None, show=True):
        cases = cases or (CASES[:5] if FAST_MODE else CASES)
        rows, passed = [], 0
        for c in cases:
            raw = generate(builder(c.text), system=SYSTEM)
            ok, problems, obj = score_one(raw, c)
            passed += ok
            rows.append({'id': c.id, 'kind': c.kind, 'pass': ok, 'problems': problems, 'raw': raw, 'obj': obj})
            if show:
                mark = 'PASS' if ok else 'FAIL'
                print(f' {c.id} [{c.kind:11}] {mark}' + (' if ok else f' <- {problems[0]}'))
        return passed, len(cases), rows

print('=== v1 · plain instruction ===')
p1, n1, rows_v1 = run_version(v1)
print(f'\npass-rate v1: {p1}/{n1}')
```

```

=== v1 · plain instruction ===
c01 [typical    ] FAIL  <- no JSON object found in the response
c02 [edge       ] FAIL  <- no JSON object found in the response
c03 [typical    ] FAIL  <- no JSON object found in the response
c04 [typical    ] FAIL  <- no JSON object found in the response
c05 [edge       ] FAIL  <- no JSON object found in the response
c06 [adversarial] FAIL  <- no JSON object found in the response
c07 [adversarial] FAIL  <- no JSON object found in the response
c08 [edge       ] FAIL  <- no JSON object found in the response
c09 [edge       ] FAIL  <- no JSON object found in the response
c10 [adversarial] FAIL  <- no JSON object found in the response

```

pass-rate v1: 0/10

Look at one failure properly

Read what the model actually returned — the fix is usually obvious once you do.

```

In [10]: _fails = [r for r in rows_v1 if not r['pass']]
         if _fails:
             f = _fails[0]
             print(f"case {f['id']} failed because:")
             for p in f['problems']:
                 print('  -', p)
             print('\n--- raw response ---')
             print(f['raw'][:800])
         else:
             print('v1 passed everything - rare. Try the 0.5B model to see failures.')

```

```

case c01 failed because:
  - no JSON object found in the response

```

```

--- raw response ---

```

```

Applicant Details:

```

- Name: Priya Nair
- Date of Birth: 1991-07-02
- Employer: Sembcorp Industries
- Monthly Income: SGD 6,500
- Loan Requested: SGD 120,000 over 6 years

6 · The climb — v1 → v4

One change per version. Watch which **rung** fixes which **kind** of failure.

```
In [11]: results = []
for name, change, builder in VERSIONS:
    print(f'=== {name} · {change} ===')
    p, n, rows = run_version(builder, show=False)
    fails = [r['id'] for r in rows if not r['pass']]
    print(f'  pass-rate {p}/{n}' + (f'  still failing: {fails}' if fails else '  all pass'))
    results.append({'version': name, 'change': change, 'passed': p, 'n': n, 'failing': fails, 'rows': rows})

print('\n' + '=' * 58)
print(f"{'version':8} {'the one change':26} {'pass':>7}")
print('-' * 58)
for r in results:
    print(f"{'version':8} {'change':26} {'passed':>3}/{r['n']:<3}")
print('=' * 58)
```

```
=== v1 · plain instruction ===
  pass-rate 0/10  still failing: ['c01', 'c02', 'c03', 'c04', 'c05', 'c06', 'c07', 'c08', 'c09', 'c10']
=== v2 · + strict JSON schema ===
  pass-rate 4/10  still failing: ['c01', 'c02', 'c03', 'c04', 'c05', 'c06']
=== v3 · + 2 few-shot examples ===
  pass-rate 10/10  all pass
=== v4 · + explicit rules ===
  pass-rate 9/10  still failing: ['c04']
```

```
=====
version  the one change                pass
-----
v1      plain instruction              0/10
v2      + strict JSON schema          4/10
v3      + 2 few-shot examples         10/10
v4      + explicit rules              9/10
=====
```

What to notice

- **v1 → v2** fixes *shape* failures (prose, wrong keys) — that is rung 4, structured output.
- **v2 → v3** fixes *consistency* (date formats, "k" amounts) — rung 2, examples doing the teaching.
- **v3 → v4** fixes the *judgement* cases (blank employer, annual salary, the co-applicant) — those needed an explicit rule, because no amount of examples tells the model your policy.

Your exact numbers will differ from any slide — different model, different day. **The shape of the climb is the lesson**, not the numbers.

7 · YOUR TURN (pair build)

In pairs: write **one new eval case** for a failure we have not covered, add it, and re-run v4. Did the pass-rate hold — or did you break it?

Ideas: a name with no surname · income in RMB or INR · a future-dated birth date · two loan amounts on one form · a form in Chinese · an employer that is also a person's name.

```
In [ ]: MY_CASE = EvalCase(
    id='c11',
    kind='edge',
    note='TODO: describe what you are testing',
    text="""Name: Siti
DOB: 2031-01-01
Employer: Self-employed
Monthly income: MYR 4,000
Loan: MYR 50,000""",
    expect={'currency': 'MYR', 'monthly_income': 4000},    # what MUST be true of a correct answer
)

ok, problems, obj = score_one(generate(v4(MY_CASE.text), system=SYSTEM), MY_CASE)
print('PASS' if ok else 'FAIL')
for p in problems:
    print('  -', p)
print(json.dumps(obj, indent=2, ensure_ascii=False) if obj else '(no JSON parsed)')
```

Note the trap in the example above: a **birth date in 2031**. Nothing in our L1 checks says a date of birth must be in the past — so a nonsense value passes. That is the real lesson of the pair build: **a pass-rate is only ever as good as the eval set behind it.**

8 · What did that cost? (bridge to Capsule 3)

Every technique on the ladder has a price tag. Few-shot examples are re-sent on **every** call; rules make the prompt longer; chain-of-thought and self-consistency multiply the output tokens.

```
In [12]: print(f"tokens this session   in: {TOKENS_USED['in']:,}   out: {TOKENS_USED['out']:,}")

# TRY THIS (knob): put your own provider's prices in and see what this run would cost at scale.
PRICE_IN_PER_M, PRICE_OUT_PER_M = 0.15, 0.60      # US$ per 1M tokens (example: a small hosted model)
run_cost = TOKENS_USED['in']/1e6*PRICE_IN_PER_M + TOKENS_USED['out']/1e6*PRICE_OUT_PER_M
print(f'this run:                about US${run_cost:.4f}')

calls = max(1, len(VERSIONS) * (5 if FAST_MODE else len(CASES)))
per_call_in, per_call_out = TOKENS_USED['in']/calls, TOKENS_USED['out']/calls
cost_per_call = per_call_in/1e6*PRICE_IN_PER_M + per_call_out/1e6*PRICE_OUT_PER_M
print(f'per call:                ~{per_call_in:,.0f} in / {per_call_out:,.0f} out tokens')
print(f'at 1,000,000 calls:      about US${cost_per_call*1_000_000:,.0f}')

# and the same sum for the MOST expensive prompt version only (v4 = rules + examples)
v4_in = len(SYSTEM + v4(SAMPLE_FORM)) / 4        # ~4 chars per token, good enough for a class estimate
print(f'v4 prompt alone:        ~{v4_in:,.0f} input tokens on EVERY call'
      f' -> US${v4_in/1e6*PRICE_IN_PER_M*1_000_000:,.0f} per 1M calls, before any output')
```

```
tokens this session   in: 11,225   out: 2,952
this run:              about US$0.0035
per call:              ~281 in / 74 out tokens
at 1,000,000 calls:    about US$86
v4 prompt alone:       ~405 input tokens on EVERY call -> US$61 per 1M calls, before any output
```

Notice that **v4 — the best prompt — is also the most expensive per call**: the rules and the two worked examples are re-sent every single time. Hold that thought; Capsule 3 puts a real model's recipe on the meter.

9 · Knobs — play with these after class

This notebook is **yours to keep**. Every experiment below is a single cell you can run and re-run. The point is not to get 10/10; it is to *feel* which lever does what.

#	Knob	The question it answers
A	temperature	why do I get a different answer each time?
B	rule ablation	which of my rules is actually earning its keep?
C	number of examples	is few-shot worth the tokens?
D	self-consistency	can I buy reliability with money?
E	the schema	what happens when the business asks for one more field?
F	your own form	does any of this survive contact with <i>my</i> documents?

A · Temperature — the randomness dial

Run this twice. At `temperature=0` the two runs are identical; at `temperature=1.0` they usually are not. **This is why we fix temperature at 0 before comparing prompts** — otherwise you cannot tell a real improvement from luck.

```
In [13]: # TRY THIS (knob): change TEMP to 0.0, 0.7, 1.5 and re-run a few times.
TEMP = 1.0
for i in (1, 2):
    out = generate(v4(CASES[1].text), system=SYSTEM, temperature=TEMP)
    obj, _ = parse_json(out)
    print(f'run {i}:', json.dumps(obj, ensure_ascii=False) if obj else out[:120])
```

```
run 1: {"full_name": "Tan Wei Ming", "date_of_birth": "1994-03-14", "employer": null, "monthly_income": 4200, "currency": "SGD", "loan_amount": 80000}
run 2: {"full_name": "Tan Wei Ming", "date_of_birth": "1994-03-14", "employer": null, "monthly_income": 4200, "currency": "SGD", "loan_amount": 80000}
```

B · Rule ablation — which rule is doing the work?

`v4` won because of seven explicit rules. But are all seven needed? Remove one and see what breaks. (Ablation = "take one piece out and re-measure" — a standard way to find out what matters.)

```
In [ ]: # TRY THIS (knob): change DROP to the text of any rule line you want to remove.
DROP = 'If income is given ANNUALLY, divide by 12 to get monthly_income.'

RULES_ABLATED = '\n'.join(l for l in RULES.split('\n') if DROP not in l)
def v4_ablated(form):
    return v4(form).replace(RULES, RULES_ABLATED)

p, n, rows = run_version(v4_ablated, show=False)
broke = [r['id'] for r in rows if not r['pass']]
print(f'v4 without that rule: {p}/{n} now failing: {broke or "nothing – the rule was not needed"}')
```

C · How many examples are worth it?

Few-shot examples cost tokens on **every single call**. Do two earn their keep over one — or zero?

```
In [ ]: # TRY THIS (knob): 0, 1 or 2 examples.
def v_examples(n_examples):
    ex = '' if n_examples == 0 else EXAMPLES.split('EXAMPLE 2')[0] if n_examples == 1 else EXAMPLES
    def build(form):
        return ("Extract the applicant details from this loan application form.\n"
                "Return ONLY a JSON object with exactly these keys:\n"
                f"  {' '.join(SCHEMA_FIELDS)}\n\n{RULES}\n\n{ex}\n"
                f"NOW DO THIS ONE\nFORM:\n{form}\nJSON:")
    return build

for k in (0, 1, 2):
    before = TOKENS_USED['in']
    p, n, _ = run_version(v_examples(k), show=False)
    print(f'{k} example(s): {p}/{n} (+{TOKENS_USED["in"]-before:,} input tokens for the run)')
```

D · Self-consistency — buying reliability with money (rung 6)

Ask the same question **n** times at a non-zero temperature, then keep the answer that appears most often. Accuracy usually rises; your bill rises by exactly nx . This is the technique Med-PaLM 2 stacked — and the one we price in Capsule 3.

```
In [12]: # TRY THIS (knob): N_SAMPLES = 1, 3, 5.
from collections import Counter
```

```

N_SAMPLES = 3
case = CASES[6]          # the annual-salary case – genuinely hard

votes = []
for _ in range(N_SAMPLES):
    obj, _err = parse_json(generate(v3(case.text), system=SYSTEM, temperature=0.8))
    if obj: votes.append(json.dumps(obj, sort_keys=True, ensure_ascii=False))

if votes:
    winner, count = Counter(votes).most_common(1)[0]
    print(f'{len(set(votes))} distinct answers from {N_SAMPLES} samples; the majority ({count}/{N_SAMPLES}) was:')
    print(json.dumps(json.loads(winner), indent=2, ensure_ascii=False))
    print(f'\ncost: {N_SAMPLES}x a single call, for one answer.')
else:
    print('no parseable answers – try N_SAMPLES=5 or a stronger model')

```

1 distinct answers from 3 samples; the majority (3/3) was:

```

{
  "currency": "SGD",
  "date_of_birth": "1993-08-22",
  "employer": "Singtel",
  "full_name": "Sofia Rahman",
  "loan_amount": 60000,
  "monthly_income": 8000
}

```

cost: 3x a single call, for one answer.

E · Change the schema — the request every builder gets

The business wants the loan **tenor** too. Add a field and watch what it costs you: the schema, the examples, the assertions and the eval cases all have to move together.

In [13]: `# TRY THIS (knob): add / remove fields here, then re-run.`
`EXTRA_FIELD = 'loan_tenor_years'`

```

def v5(form):
    return (f"Extract the applicant details from this loan application form.\n"
            f"Return ONLY a JSON object with exactly these keys:\n"

```

```

        f" {' '.join(SCHEMA_FIELDS + [EXTRA_FIELD])}\n\n{RULES}"
        f"- {EXTRA_FIELD} is the loan period in years as a number, or null if not stated.\n\n"
        f"FORM:\n{form}\nJSON:")

obj, err = parse_json(generate(v5(CASES[0].text), system=SYSTEM))
print(json.dumps(obj, indent=2, ensure_ascii=False) if obj else err)
print('\n^ note: our l1_checks would now FAIL this for an "unexpected key".')
print(' Changing the output shape means changing the tests too – that is the real cost of a new field.')

{
  "full_name": "Priya Nair",
  "date_of_birth": "1991-07-02",
  "employer": "Sembcorp Industries",
  "monthly_income": 6500,
  "currency": "SGD",
  "loan_amount": 120000,
  "loan_tenor_years": 6
}

^ note: our l1_checks would now FAIL this for an "unexpected key".
  Changing the output shape means changing the tests too – that is the real cost of a new field.

```

F · Bring your own document

Paste anything with fields in it — an invoice, a CV, a medical intake form, a purchase order. Change the schema to match, and you have just built a different product with the same method.

```

In [ ]: # TRY THIS (knob): paste your own text (SYNTHETIC / non-confidential only).
MY_FORM = """Name: Your Name Here
DOB: 1 Jan 2000
Employer: Somewhere Pte Ltd
Income: 5k per month
Loan: 60k"""

obj, err = parse_json(generate(v4(MY_FORM), system=SYSTEM))
print(json.dumps(obj, indent=2, ensure_ascii=False) if obj else err)

```

Recap

1. **Structured output** turned an answer into a **component** a program can use.
2. **L1 assertions** turned "seems better" into a **number**.
3. **One change per version** made every improvement **attributable** to a technique.
4. Your eval set is your ruler — and it only measures what you thought to put in it.

This is Assignment 1, part (iii): a structured-output prompt, a 10-case eval harness, and your pass-rate history. Due end of Week 4.